NoteFlow

Product Requirements Document

Personal Notes App with AI Summarization — TickTick-Inspired UI

Version	Stack	Status
1.0.0	Next.js + Supabase + Claude AI	Planning Phase

1. Project Overview

NoteFlow is a personal notes application with AI-powered summarization. The app is built with a clean three-panel layout inspired by TickTick — sidebar for navigation, note list in the middle, and the active note editor on the right.

This is a return-to-coding project. Every line of code is written manually. The goal is to touch every layer of a modern full-stack application: auth, database, server components, client interactivity, and external AI API integration.

1.1 Goals

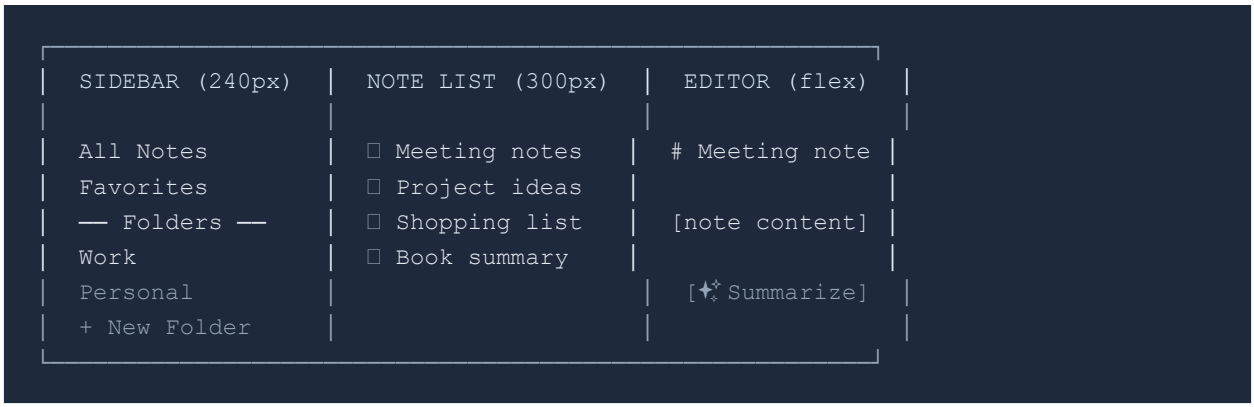
- Build a complete, working full-stack notes app from scratch
- Write all code manually — AI used only as search engine and final reviewer
- Implement a real AI feature using the Anthropic API (note summarization)
- Replicate the TickTick UI pattern: sidebar + list panel + editor panel
- Deploy to Vercel and use it as a personal level assessment tool

1.2 Non-Goals

- Not a portfolio project — this is a private return-to-form exercise
- No real-time collaboration, no sharing, no public notes
- No mobile app — responsive web only
- No over-engineering — finish it, then move on

2. UI Layout — TickTick-Inspired

The interface is divided into three panels, exactly like TickTick. This layout is the defining UX pattern of the entire app.



2.1 Panel Breakdown

Panel	Width	Responsibilities
Sidebar	240px fixed	Navigation: All Notes, Favorites, Folders list, New Folder button, user avatar + sign out
Note List	300px fixed	Scrollable list of notes in selected folder. Note title, preview snippet, date, tag chips. New note button at top.
Editor	flex (fills rest)	Active note editor. Title input, tag input, rich text body, and the AI Summarize button + summary display area.

3. Technology Stack

Layer	Technology	Purpose
Framework	**Next.js 14 (App Router)**	Server components, routing, API routes
Language	**TypeScript**	Type safety across the full stack
Styling	**Tailwind CSS**	Utility classes, fast layout iteration
Components	**shadcn/ui**	Accessible primitives you wire

		manually
Database + Auth	**Supabase**	PostgreSQL + Auth + RLS in one platform
AI Summarization	**Anthropic Claude API**	Summarize note content on demand
Deployment	**Vercel**	Zero-config Next.js hosting
Version Control	**GitHub**	Source of truth

4. Feature Requirements

4.1 Authentication

Feature	Description	Priority
Sign Up	Email + password via Supabase Auth	**Must Have**
Sign In	Email + password with persistent session	**Must Have**
Sign Out	Clear session, redirect to login page	**Must Have**
Protected Routes	Middleware blocks unauthenticated access	**Must Have**

4.2 Notes

Feature	Description	Priority
Create Note	New note with title, body, optional tags	**Must Have**
Edit Note	Auto-save or manual save while editing	**Must Have**
Delete Note	Delete with confirmation, removes from list	**Must Have**
Note Preview	List panel shows title + first 80 chars of body	**Must Have**
Note Timestamps	Created at and last updated shown in editor	**Must Have**
Favorite Note	Star a note, appears in Favorites section	**Should Have**
Search Notes	Search bar filters note list by title or content	**Should Have**

4.3 Folders

Feature	Description	Priority
Create Folder	Named folder via sidebar button	**Must Have**
Rename Folder	Inline rename on double click	**Should Have**
Delete Folder	Delete folder and move notes to All Notes	**Should Have**
All Notes View	Default view showing every note across folders	**Must Have**
Favorites View	View showing only favorited notes	**Should Have**

4.4 Tags

Feature	Description	Priority
Add Tags	Type tag name in editor, press Enter to add	**Should Have**
Remove Tags	Click X on tag chip to remove	**Should Have**
Tag Display	Tag chips shown in note list preview and editor	**Should Have**

4.5 AI Summarization — Core Feature

Feature	Description	Priority
Summarize Button	Button in editor triggers Claude API call	**Must Have**
Loading State	Spinner + disabled state while AI is processing	**Must Have**
Summary Display	Summary rendered below the note in a styled card	**Must Have**
Regenerate	Button to regenerate a new summary	**Should Have**
Save Summary	Option to append summary to the note body	**Nice to Have**
Error Handling	Clear message if API call fails	**Must Have**

4.6 UX & Polish

Feature	Description	Priority
Empty States	Friendly message when folder has no notes	**Must Have**
Loading Skeletons	Skeleton placeholders during data fetch	**Should Have**
Error Boundaries	User-facing error messages on failures	**Must Have**
Responsive Layout	Collapses panels gracefully on smaller screens	**Should Have**
Dark Mode	Theme toggle, persisted in localStorage	**Nice to Have**
Keyboard Shortcut	Ctrl+N to create new note	**Nice to Have**

5. Database Schema

Design this on paper first. The schema decisions you make here will be part of the level assessment. Think about relationships, nullability, and RLS before touching Supabase.

Table: users

Column	Type	Constraints	Description
`id`	`uuid`	PK	Managed by Supabase Auth automatically
`email`	`text`	NOT NULL, UNIQUE	User email address
`created_at`	`timestamptz`	default now()	Account creation time

Table: folders

Column	Type	Constraints	Description
`id`	`uuid`	PK, default gen	Unique folder identifier
`user_id`	`uuid`	FK → users.id	Owner of this folder
`name`	`text`	NOT NULL	Folder display name
`created_at`	`timestamptz`	default now()	Creation timestamp

Table: notes

Column	Type	Constraints	Description

id	uuid	PK, default gen	Unique note identifier
user_id	uuid	FK → users.id	Owner (used in RLS policy)
folder_id	uuid	FK → folders.id, nullable	Parent folder, null = All Notes
title	text	NOT NULL, default ''	Note title
body	text	NOT NULL, default ''	Note content
is_favorite	boolean	default false	Favorited by user
ai_summary	text	nullable	Last generated AI summary
created_at	timestamptz	default now()	Creation timestamp
updated_at	timestamptz	default now()	Last update (use trigger)

Table: tags

Column	Type	Constraints	Description
id	uuid	PK, default gen	Unique tag identifier
user_id	uuid	FK → users.id	Owner of this tag
name	text	NOT NULL	Tag label text
color	text	nullable	Optional hex color

Table: note_tags

Column	Type	Constraints	Description
note_id	uuid	FK → notes.id	Reference to note
tag_id	uuid	FK → tags.id	Reference to tag
PRIMARY KEY	(note_id, tag_id)	composite	Prevents duplicate tag assignments

RLS Rule: Enable Row Level Security on ALL tables. Every policy should follow: user_id = auth.uid(). Do this before any data is written. Never skip this step.

6. Project Folder Structure

```
src/
  app/
    (auth)/
      login/page.tsx
```

```
    signup/page.tsx
  (dashboard)/
    layout.tsx          ← three-panel layout wrapper + auth check
    page.tsx            ← defaults to All Notes view
    folder/[id]/page.tsx
    favorites/page.tsx
components/
  ui/                   ← shadcn components (don't touch these)
  layout/
    Sidebar.tsx         ← left panel: folders + navigation
    NoteList.tsx        ← middle panel: list of notes
    NoteEditor.tsx      ← right panel: active note editor
    AppShell.tsx        ← three-panel layout container
  notes/
    NoteCard.tsx        ← single note in the list
    NoteForm.tsx        ← title + body inputs
    AISummary.tsx       ← summarize button + result display
  folders/
    FolderItem.tsx
    NewFolderButton.tsx
lib/
  supabase/
    client.ts           ← browser Supabase client
    server.ts           ← server Supabase client
  anthropic.ts          ← Claude API call for summarization
  types.ts              ← shared TypeScript interfaces
  utils.ts              ← formatDate, truncate, etc.
hooks/
  useNotes.ts
  useFolders.ts
middleware.ts           ← auth protection for dashboard routes
```

7. AI Summarization — Implementation Notes

7.1 How It Works

- User opens a note and clicks the Summarize button in the editor
- Frontend sends a POST request to /api/summarize with the note body
- Next.js API route calls the Anthropic Claude API with the note content
- Claude returns a concise summary (2-4 sentences)
- Summary is displayed in a styled card below the note editor
- Summary is optionally saved to the ai_summary column in Supabase

7.2 API Route

```
// app/api/summarize/route.ts
```

```
POST /api/summarize
Body: { noteId: string, content: string }

1. Validate that content is not empty
2. Call Anthropic API with prompt:
   'Summarize the following note in 2-4 clear sentences:'
3. Return: { summary: string }
4. Handle errors: empty content, API failure, rate limits
```

Important: Never expose your Anthropic API key on the client. The call must go through a Next.js API route (server-side only). Store the key in .env.local as ANTHROPIC_API_KEY.

8. Build Milestones

One milestone per day. Do not start the next until the current one works end-to-end.

Day	Milestone	Tasks	Phase
Day 1	Project Setup	Init Next.js + TypeScript, install Tailwind + shadcn + Supabase SDK, push to GitHub	Setup
Day 2	DB Schema	Create all tables in Supabase dashboard, enable RLS, write policies, test with SQL editor	Setup
Day 3	Auth	Sign up, sign in, sign out pages. Middleware protection. Session persistence.	Core
Day 4	Three-Panel Layout	Build AppShell with Sidebar + NoteList + NoteEditor. Static data, no DB yet.	Core
Day 5	Folders CRUD	Create, rename, delete folders. Sidebar reflects real Supabase data.	Core
Day 6	Notes CRUD	Create, edit, delete, display notes. NoteList updates in real time.	Core
Day 7	Favorites + Search	Toggle favorite, Favorites view, search bar filtering note list.	Core
Day 8	AI Summarization	API route, Anthropic integration, Summarize button, summary display card.	AI
Day 9	Polish	Empty states, loading skeletons, error handling, responsive layout.	Polish
Day 10	Deploy	Vercel deploy, environment variables, final	Polish

		QA, submit for review.	

9. Development Rules

Your Rules (Written Down)

- Write every line of code yourself — no copy-paste from AI outputs
- Copilot autocomplete is allowed — it is not cheating
- Use AI only as a search engine — to understand concepts, not to generate solutions
- When you hit a bug, debug it yourself first before asking anything
- Read every line you write — if you don't understand it, rewrite it until you do
- No feature is done until the error path is handled, not just the happy path
- Commit after every milestone with a meaningful message
- TypeScript strict mode — no 'any' types

10. Level Assessment Criteria

When the project is complete, bring the full codebase for an honest review. These are the exact criteria that will be evaluated.

Criterion	What It Reveals About Your Level
DB Schema	Correct relationships, nullable fields, RLS policies, use of triggers for updated_at
Component Design	Are components focused and composable, or bloated and tangled?
TypeScript	Proper interfaces, no 'any', types shared across client and server
Server vs Client	Correct use of Server Components vs 'use client' — no unnecessary client components
Error Handling	Errors surfaced to users, not swallowed silently in catch blocks
AI Integration	API key protected server-side, loading/error states handled correctly
State Management	State lives in the right place — no prop drilling, no over-engineering
Code Readability	Would another developer understand this without asking you questions?

The Only Rule That Matters

There is only one goal: keep learning and improving. Titles don't matter. The code you write here will tell the truth about where you are — and where you're going.